Read and Process a Text File

Project Report — Text File Statistics Web Application

1. Project Overview

This project implements a Python-based text-processing application that reads a UTF-8 text file and generates useful statistics: line count, word count, and character count. A Flask web interface allows a user to upload a .txt file and view the generated results in a browser. The same processing logic is also available through a command-line interface.

2. Objective

The main objective is to demonstrate file handling and text processing in Python, while applying good practices such as context managers, exception handling, incremental processing, testing, Git/GitHub version control, and cloud deployment.

3. Key Features

Feature	Implementation
Text file upload	Browser-based upload of UTF-8 .txt files
Line count	Counts lines while streaming through the file
Word count	Uses whitespace splitting for each line
Character count	Counts characters excluding line-ending characters
Error handling	Handles missing files, wrong extensions, invalid UTF-8 and oversized uploads
Large-file consideration	Processes content line-by-line instead of loading the whole file
CLI support	process a local file with text_stats.py
Testing	Automated unit tests for normal and empty files
Deployment	Render-ready Flask Web Service configuration

4. Technologies Used

Python: core programming and file processing.
Flask: web application and file-upload interface.
Gunicorn: production WSGI server for deployment.
HTML/CSS: responsive user interface.
Git/GitHub: source control and project submission.
Render: cloud deployment platform.

5. File Handling Approach

The local file processor uses a context manager (`with open(...)`) so the file is properly closed even when an error occurs. The application reads the file line-by-line. This avoids unnecessarily loading the entire contents into memory and is appropriate for larger text files. UTF-8 is used as the default encoding, and invalid text is reported to the user rather than producing incorrect statistics.

6. Counting Logic

Lines: each line yielded by the text stream increments the line count.
Words: `line.split()` separates words using Python whitespace rules.
Characters: the length of each line after removing `\r` and `\n` is added to the character count.
Empty file: all three statistics are zero.

7. Error Handling

The application validates that a file was selected and that its extension is .txt. It also enforces a 10 MB upload limit, catches UTF-8 decoding failures, and handles file read errors gracefully. The web application exposes `/health` as a simple deployment health-check endpoint.

8. Testing

Automated tests cover a normal text file and an empty file. The recommended command is `python -m unittest discover -s tests -v`. Manual browser testing should additionally verify file upload, invalid extension handling, invalid UTF-8 handling, and the deployed health endpoint.

9. Deployment

The project is configured for deployment as a Render Python Web Service. The build command installs dependencies from requirements.txt, while the start command runs `gunicorn app:app`. The included render.yaml provides the same configuration in infrastructure-as-code form.

10. GitHub and Submission

The project includes a README, deployment guide, submission checklist, Render configuration, source code, tests, sample input, and this report. The intended submission consists of the public GitHub repository URL, the live Render URL, the PDF report, and deployment/rollback evidence when requested by the internship portal.

11. Conclusion

The completed project demonstrates practical Python file handling and text processing through a small but deployable web application. It combines clean separation of processing logic from the web layer, graceful error handling, memory-conscious processing, automated testing, version control, and cloud deployment.